



deti

universidade de aveiro
departamento de electrónica,
telecomunicações e informática

Project 2: DjanGoMarket

TPW

Vítor Alves - 104296
Filipe Viseu - 119192
Samuel Vinhas - 119405

2025/2026

Contents

1	DjanGoMarket - Supermarket Management System	1
1.1	Introduction	1
2	Main Features	2
2.1	REST API (Django REST Framework)	2
2.2	Angular SPA	2
2.3	User Authentication & Authorization	3
3	Access Information	4
3.1	Deployed Application	4
3.2	User Roles & Permissions	4
3.2.1	CEO (Admin)	4
3.2.2	Manager	4
3.2.3	Cashier	4
3.2.4	Employee	4
3.3	Demo Accounts	4
4	Configuration for Running Locally	5
4.1	Prerequisites	5
4.2	Installation Steps	5
5	Conclusions	6
5.1	What Went Well	6
5.2	Limitations	6
5.3	What We'd Improve	6
5.4	Final Thoughts	6

1. DjanGoMarket - Supermarket Management System

1.1 Introduction

DjanGoMarket TP2 extends the system built in TP1 into an n-tier architecture. The Django backend now exposes a **Django REST Framework** API, and the frontend is rewritten as an **Angular 21** single-page application. All TP1 models, business logic, and the role-based permission system are preserved - TP2 adds a REST layer on top and replaces server-rendered HTML with a proper SPA consuming JSON over HTTP. The system continues to cover the same supermarket management domain:

- Employee and personnel management
- Product inventory and warehouse organization
- Customer purchases and transactions
- Supplier/Distributor relationships
- Store locations and section management

2. Main Features

2.1 REST API (Django REST Framework)

- Full CRUD via `ModelViewSet` for all 9 entities: Supermarket, Section, Employee, Product, Warehouse, Distributor, Client, Purchase, Order
- **JWT authentication** via `django-rest-framework-simplejwt` - login with employee number and password, receive access + refresh tokens
- **Role-based permissions** using `DjangoModelPermissions` mapped to the same four Django groups (CEO / Manager / Cashier / Employee)
- **Scoped querysets** - non-CEO users only see data from their own supermarket
- **Search and ordering filters** on every endpoint (e.g., search employees by name/role, order purchases by date)
- **Nested serializers** - purchases and orders include their line items on read; writes use a flat `item_data` list
- `/api/me/` - current user profile including group, supermarket, and supervisor
- `/api/me/ PATCH` - self-service profile editing (name, contact, age, sex)
- `/api/me/password/` - authenticated password change
- `/api/health/` - unauthenticated health check

2.2 Angular SPA

- **Angular 21** with standalone components and lazy-loaded routes
- Feature-based folder structure: **features/** (one folder per entity), **core/** (services, guards, interceptors, models), **shared/** (navbar)
- Full CRUD pages for all 9 entities: list, detail, create/edit form
- **AuthService** - manages JWT tokens in `localStorage`, exposes reactive user state via `BehaviorSubject`

- **tokenInterceptor** - automatically attaches **Authorization: Bearer** header on every request; silently refreshes the access token on 401 and retries; redirects to login if refresh also fails
- **authGuard** - blocks all routes except `/login` when no valid token is present
- **Role-based navigation** - navbar filters entries to only what the current user's group can access
- **Profile page** - view own employee details, edit personal info, change password

2.3 User Authentication & Authorization

Login sends `{enumber, password}` to `/api/token/` and stores the returned JWT pair. All API requests carry **Authorization: Bearer <access_token>**; expired tokens are refreshed transparently in the background. The system uses the same four-group model as TP1, enforced at API level via Django model permissions:

- **CEO** - full CRUD across all supermarkets; `is_staff=True`, admin panel access
- **Manager** - full CRUD scoped to their supermarket; read-only on products, sections, distributors
- **Cashier** - create purchases; read-only elsewhere
- **Employee** - read-only across their assigned supermarket

Non-CEO users cannot edit or delete CEO employees (enforced in `EmployeeViewSet`).

3. Access Information

3.1 Deployed Application

The application is deployed at:

- Backend API: <https://djangumarket.pythonanywhere.com/api/>
- Angular UI: <https://djangumarket.vercel.app/>

3.2 User Roles & Permissions

The system uses Django's group-based permission system to control user access.

3.2.1 CEO (Admin)

Full system access and control. View all data across all supermarkets; full CRUD on all entities. `is_staff=True`, can access admin panel. **Data Scope:** Global - all supermarkets and data.

3.2.2 Manager

Supermarket-level management. Full CRUD on employees, warehouses, purchases, orders within their supermarket. Read-only on products, sections, distributors. **Data Scope:** Limited to their assigned supermarket.

3.2.3 Cashier

Create purchases (point of sale transactions). View purchase history, products, orders. **Data Scope:** Transaction-related data only.

3.2.4 Employee

View-only access to company data. Cannot create, edit, or delete any records. **Data Scope:** Limited to their supermarket (view-only).

3.3 Demo Accounts

Role	Username (enumber)	Password
CEO (Admin)	1000	password123
Manager	1001	password123
Cashier	1002	password123
Employee	1005	password123

4. Configuration for Running Locally

4.1 Prerequisites

- Python 3.8+
- Node.js 18+
- Git

4.2 Installation Steps

1. Clone the project:

```
git clone https://github.com/samuelvinhas/DjanGoMarket.git
cd DjanGoMarket
git checkout angular
```

2. Backend setup:

```
cd django
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
./make-env.sh
./migrate.sh
python3 setup_groups.py
python3 populate_db.py # optional
python3 manage.py runserver
```

3. Frontend setup (new terminal):

```
cd angular
npm install
npm start
```

- Angular UI: <http://localhost:4200>
- API: <http://localhost:8000/api/>

5. Conclusions

5.1 What Went Well

Splitting the backend and frontend into separate tiers was cleaner than expected. DRF's `ModelViewSet` covers most of the boilerplate for a CRUD API, and the existing Django group/permission model mapped directly onto DRF's permission classes without any changes to how groups work. The Angular interceptor handling silent token refresh meant the rest of the app could ignore token expiry entirely.

Reusing all models and business logic from TP1 without touching them was a good call - it kept the scope of TP2 focused on the new layer rather than revisiting old ground.

5.2 Limitations

The biggest limitation remains the default password setup for employees - obviously not something we would ship in a real product, but as this was not the main focus of the project, we went with a simple approach. In the previous project (TP1) we had no option to change that password, to make this limitation less severe, in TP2 we added a password change endpoint and UI for users to update their own password after logging in with the default one.

5.3 What We'd Improve

Server-side pagination would be worth adding for larger datasets - currently all records are fetched at once. Better tests like end-to-end tests for the Angular app would also be a great addition.

5.4 Final Thoughts

Overall, DjanGoMarket does what it set out to do. Building a full supermarket management system and then converting it to a proper n-tier architecture was a great learning experience.